IPConfig

Onchain Intellectual Property Registry

Technical Whitepaper
Version 1.0 · BOT Chain

LEGAL NOTICE: This document describes a blockchain-based evidence and licensing system. Registering a work on IPConfig creates a timestamped, verifiable claim — it does NOT grant legal copyright protection, does not constitute legal proof of authorship or originality, and does not replace formal intellectual property registration under applicable law. Consult a qualified legal professional for formal IP protection.

Contents

1 Introduction & Purpose

2 The Evidence Model

2.1 What onchain registration proves

2.2 What it does not prove

2.3 The role of cryptographic hashes

3 Smart Contract Architecture

3.1 IP Registry

3.2 License Terms Manager

3.3 License Purchase Manager

3.4 Revenue Distribution

3.5 Dispute & Revocation Records

3.6 Ownership Transfer

4 Licensing Model

4.1 License terms versioning

4.2 Payment tokens — BOT and USDT

4.3 License expiry and perpetual licenses

5 Security Considerations

6 Platform Architecture

7 Limitations & Responsible Use

8 Glossary

1. Introduction & Purpose

IPConfig is an onchain intellectual property registry deployed on BOT Chain. It provides creators, rights holders, businesses, and individuals with a decentralised platform for recording IP claims, managing licensing terms, issuing licenses, and distributing licensing revenue — without relying on centralised databases, intermediaries, or traditional registration bodies.

The platform serves three primary constituencies:

- **Creators and rights holders:** Register works, declare licensing terms, issue licenses, receive revenue, and manage ownership.
- **Licensees:** Browse registered IP, purchase licenses, verify license status, and manage their license portfolio.
- **Platform administrators:** Manage platform parameters, review flagged records, and adjudicate reported disputes.

IPConfig is designed to operate transparently: all registration records, license terms versions, license purchases, and revenue accruals are stored on-chain and are publicly readable. The system enforces its rules through Solidity smart contracts — there is no trusted central party that can unilaterally alter records once submitted.

2. The Evidence Model

2.1 What onchain registration proves

When a creator registers a work on IPConfig, the system records the following on-chain at the block timestamp of the transaction:

- The title, category, and description provided by the registrant.
- The declared creation date.
- The block timestamp at registration — the moment the transaction was mined.
- A cryptographic hash of the work (SHA-256 or keccak256), computed by the registrant.
- The rights holder wallet address declared by the registrant.
- An optional metadata URI (e.g. an IPFS content identifier).

Together, this record provides evidence that a specific file (identified by its hash) existed and was associated with a specific wallet address at a known point in time. This can be valuable in legal proceedings as a component of a chain of evidence, but is not by itself a legal determination of ownership.

2.2 What it does not prove

> A blockchain registration does NOT constitute legal proof of authorship, originality, or copyright ownership under any national or international legal framework. It records a claim and an evidence reference — it does not adjudicate competing claims. Legal recognition of intellectual property rights requires compliance with applicable law and, where disputes arise, adjudication by competent legal authority.

- It does not prove the registrant created the work.
- It does not prove the work is original.
- It does not prevent another party from registering an identical hash first.
- It does not replace national copyright registration systems.
- It does not automatically grant any legal right to sue for infringement.

2.3 The role of cryptographic hashes

A cryptographic hash (SHA-256 or keccak256) is a fixed-length fingerprint of a file. Any change to the file — even a single bit — produces a completely different hash. This property means that if you can produce the original file that generates the registered hash, you can demonstrate that this exact version of the file existed before the registration block timestamp.

IPConfig rejects duplicate hashes — the same hash cannot be registered twice. This provides a simple first-mover record. However, it does not prevent a party from submitting the hash of someone else's work — the system records the claim, not its truthfulness. Disputes can be reported and reviewed by administrators.

Files are NEVER stored on-chain. Only the hash and metadata are recorded.

3. Smart Contract Architecture

IPConfig is implemented as a single Solidity contract (`IPConfig.sol`) deployed on BOT Chain. The contract manages five logical subsystems:

3.1 IP Registry

The IP registry stores `IPRecord` structs keyed by a sequential integer ID. Each record contains all registration metadata, the work hash, the rights holder address, and a flag for the currently active license terms version. A `hashToIpId` mapping enforces hash uniqueness across all registrations.

IP categories supported: Literary, Musical, Artistic, Software, Film, Patent, Trademark, and Other.

3.2 License Terms Manager

Rights holders publish `LicenseTerms` structs for each IP. Each publication increments the version counter for that IP. Once published, a terms version is immutable — it cannot be edited or deleted. The `ipTermsHistory` mapping retains all versions.

Terms encode: permitted uses (text), restrictions (text), commercial use flag, derivatives flag, attribution requirement, fees in BOT and USDT, duration in days (0 = perpetual), and publication timestamp.

> ■ KEY DESIGN PRINCIPLE: When a licensee purchases a license, the terms version ID is recorded in the License struct. If the rights holder subsequently publishes new terms, those changes do NOT affect previously issued licenses. Each licensee is permanently governed by the terms version they accepted at purchase time.

3.3 License Purchase Manager

Licenses are purchased via `purchaseLicenseWithBOT()` or `purchaseLicenseWithUSDT()`. Both functions verify that the IP is active and unflagged, that terms are set, and that the correct fee is paid. Excess BOT payments are refunded.

Each `License` struct records: IP ID, terms version ID (snapshot), licensee address, payment token, amount paid, issuance timestamp, expiry timestamp (0 for perpetual), status, and an optional note.

3.4 Revenue Distribution

Licensing fees are split between the rights holder and the platform at the time of purchase. The platform fee is set in basis points (1 bps = 0.01%; max 1000 bps = 10%). Rights-holder revenue accumulates in `pendingRevenueBOT` and `pendingRevenueUSDT` mappings and is withdrawn on-demand via `withdrawRevenueBOT()` and `withdrawRevenueUSDT()`.

3.5 Dispute & Revocation Records

Any wallet can report a dispute against an IP record. Disputes record the reporter address, reason text, and status (Open / Resolved / Dismissed). Administrators resolve disputes and may flag the associated IP, which prevents new license purchases against it.

Rights holders and administrators may revoke individual licenses by calling `revokeLicense()`. Revoked licenses are marked with `LicenseStatus.Revoked` and `isLicenseActive()` returns false.

3.6 Ownership Transfer

IP ownership may be transferred through an explicit two-step process. The current rights holder calls `proposeOwnershipTransfer(ipId, proposed)`. The proposed new owner must call `acceptOwnershipTransfer(ipId)` to complete the transfer. No unilateral silent transfers are possible.

4. Licensing Model

4.1 License terms versioning

Each time a rights holder publishes new terms for an IP, a new version is created and stored immutably. The contract records all historical versions in the `ipTermsHistory` array. The `activeTermsId` field on the IP record is updated to point to the latest version, but each License struct stores the specific terms ID that was active when the license was purchased — ensuring licensees are always governed by the agreement they accepted.

4.2 Payment tokens — BOT and USDT

Rights holders can configure fees in BOT (the native token of BOT Chain, 18 decimals) and/or USDT (ERC-20, 6 decimals). Licensees choose their preferred payment method at purchase time, subject to the rights holder having configured that token. USDT purchases require an ERC-20 approval transaction before the license purchase.

4.3 License expiry and perpetual licenses

Rights holders configure a duration in days. A duration of 0 indicates a perpetual (non-expiring) license. For timed licenses, the expiry timestamp is computed as the block timestamp at purchase plus the duration in seconds. The `isLicenseActive(licenseId)` view function checks both the status field and the current time against the expiry timestamp.

5. Security Considerations

	Mitigation
withdrawals	Revenue withdrawn by updating state before the external call. Uses call{value:} with explicit success check.
gistration of another's hash	Hash uniqueness enforced by hashToIpId mapping. First mover records. Dispute mechanism available for challenges.
date affecting licensees	Terms versions are immutable. Each License records the terms ID snapshot at purchase time. New versions cannot alter ex

ership transfer	Two-step transfer: propose + accept. The proposed party must explicitly call acceptOwnershipTransfer().
orm fee	Platform fee capped at 1000 bps (10%) in setPlatformFee().
promise	Admin address should be migrated to a multisig after deployment via transferAdmin().
nsing	Flagged IPs reject new license purchases. Admin can flag on dispute resolution.
bstitution	USDT address set at construction and updatable only by admin. Verify token address before deployment.

Before mainnet deployment, the contract should be audited by a qualified smart contract security firm. This document and the accompanying code are provided for review and are not a substitute for a professional security audit.

6. Platform Architecture

IPConfig is designed as a fully decentralised application with no backend server or centralised database. All persistent state lives on-chain.

Layer	Technology	Role
Frontend	HTML5, CSS3, Vanilla JS	Single-page app served as a static file
Wallet integration	ethers.js v6	Signs transactions, reads chain state
Smart contract	Solidity 0.8.20	IP registry, licensing, revenue, disputes
Payment — native	BOT (native token)	Direct msg.value payment
Payment — ERC-20	USDT on BOT Chain	ERC-20 approve + transferFrom
Network	BOT Chain	EVM-compatible L1/L2

7. Limitations & Responsible Use

Legal status: Blockchain registration is evidence, not legal recognition. Legal copyright exists independently of any registry.

Hash alone is not proof of authorship: Anyone can submit the hash of someone else's work. The system records first-mover claims, not authorship determination.

No file storage: Files are NEVER stored on-chain. Registrants must retain their original files to demonstrate hash correspondence.

Dispute resolution is centralised: Dispute adjudication is performed by platform administrators — a degree of trust is placed in the admin address.

Smart contract risk: As with all smart contracts, bugs or vulnerabilities may exist. Users should evaluate the audited contract code before significant use.

Blockchain permanence: Records are permanent and cannot be deleted. Users should not register sensitive personal information in descriptions or metadata.

Jurisdiction variability: IP law varies significantly by jurisdiction. This platform does not constitute legal advice and is not a substitute for professional counsel.

8. Glossary

Basis points (bps)	A unit equal to 0.01%. 300 bps = 3%.
BOT	The native currency of BOT Chain, used for gas fees and licensing payments.
Cryptographic hash	A deterministic fixed-length fingerprint of a file. SHA-256 and keccak256 are used by IPConfig.
IP record	An on-chain struct recording an IP claim, its hash, metadata, and rights holder address.
License	An on-chain record granting a licensee permission to use an IP under specific terms, for a specified duration.
License terms version	An immutable snapshot of the licensing conditions for an IP at a point in time.
Perpetual license	A license with no expiry date (durationDays = 0).
Rights holder	The wallet address declared by the registrant as the holder of rights over an IP record.
USDT	A USD-pegged stablecoin ERC-20 token; accepted as a licensing payment token alongside BOT.
Work hash	The SHA-256 or keccak256 hash of the original work file, submitted by the registrant at registration time.